

## Code:

```
GNU nano 8.7
#include <stdio.h>

#define P 5    // Processes
#define R 4    // Resources

int main() {

    int i, j, k;

    // Allocation Matrix
    int allocation[P][R] = {
        {0, 1, 0, 1}, // Google Drive
        {2, 0, 0, 0}, // Firefox
        {3, 0, 2, 1}, // Word Processor
        {2, 1, 1, 0}, // Excel
        {0, 0, 2, 0}  // PowerPoint
    };

    // Maximum Matrix
    int max[P][R] = {
        {7, 5, 3, 4},
        {3, 2, 2, 2},
        {9, 0, 2, 2},
        {2, 2, 2, 2},
        {4, 3, 3, 3}
    };

    // Available Resources
    int available[R] = {3, 3, 2, 2};

    int need[P][R];
    int finish[P] = {0};
    int safeSeq[P];
    int count = 0;

    char *processNames[P] = {
        "Google Drive",
        "Firefox",
        "Word Processor",
        "Excel",
        "PowerPoint"
    };

    // Calculate Need Matrix
    for(i = 0; i < P; i++)
        for(j = 0; j < R; j++)
            need[i][j] = max[i][j] - allocation[i][j];

    // Banker's Algorithm
    while(count < P) {
        int found = 0;

        for(i = 0; i < P; i++) {
            if(finish[i] == 0) {
```



GNU nano 8.7

```
char *processNames[P] = {
    "Google Drive",
    "Firefox",
    "Word Processor",
    "Excel",
    "PowerPoint"
};

// Calculate Need Matrix
for(i = 0; i < P; i++)
    for(j = 0; j < R; j++)
        need[i][j] = max[i][j] - allocation[i][j];

// Banker's Algorithm
while(count < P) {
    int found = 0;

    for(i = 0; i < P; i++) {
        if(finish[i] == 0) {
            int flag = 1;

            for(j = 0; j < R; j++) {
                if(need[i][j] > available[j]) {
                    flag = 0;
                    break;
                }
            }

            if(flag) {
                for(k = 0; k < R; k++)
                    available[k] += allocation[i][k];

                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0) {
        printf("\nSystem is NOT in SAFE state.\n");
        return 0;
    }
}

printf("\nSystem is in SAFE state.\nSafe Sequence:\n");
for(i = 0; i < P; i++)
    printf("%s -> ", processNames[safeSeq[i]]);
printf("END\n");

return 0;
}
```



```
john@DESKTOP-JA0TCLF MINGW64 ~  
$ gcc --version  
gcc (GCC) 15.2.0  
Copyright (C) 2025 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions. There is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

```
john@DESKTOP-JA0TCLF MINGW64 ~  
$ nano bankers.c
```

```
john@DESKTOP-JA0TCLF MINGW64 ~  
$ gcc bankers.c -o bankers
```

```
john@DESKTOP-JA0TCLF MINGW64 ~  
$ ./bankers
```

```
System is in SAFE state.  
Safe Sequence:  
Firefox -> Excel -> Word Processor -> PowerPoint -> Google Drive -> END
```

```
john@DESKTOP-JA0TCLF MINGW64 ~  
$ |
```